

Documentación técnica breve

Veterinaria App - Sistema de gestión de clínica veterinaria desarrollado como proyecto integrador para Laboratorio de Software.

1. Descripción del sistema

La aplicación permite administrar la información básica de una clínica veterinaria. El sistema incluye gestión de dueños, mascotas y turnos de atención. Cada turno se asocia a una mascota y calcula automáticamente un costo estimado según el motivo de la consulta.

El objetivo fue construir una aplicación simple, funcional y defendible, aplicando Java, Spring Boot, arquitectura MVC, JPA/Hibernate, base de datos H2 y conceptos de Programación Orientada a Objetos.

2. Tecnologías utilizadas

Tecnología	Uso en el sistema
Java 17	Lenguaje principal del backend.
Spring Boot 4.0.6	Framework para crear la aplicación, configurar componentes y ejecutar el servidor embebido.
Spring Web MVC	Creación de controllers REST y manejo de solicitudes HTTP.
Spring Data JPA	Repositorios para persistencia de datos mediante JpaRepository.
Hibernate	Implementación de JPA que convierte entidades Java en registros de base de datos.
H2 Database	Base de datos relacional en memoria para pruebas y presentación.
HTML, CSS y JavaScript	Interfaz web simple para consumir la API REST.

3. Arquitectura aplicada

El proyecto usa una arquitectura monolítica organizada bajo el patrón MVC y separada en capas:

Capa	Responsabilidad
Controller	Recibe solicitudes HTTP, interpreta parámetros y cuerpos JSON, y devuelve respuestas JSON.
Service	Contiene la lógica de negocio, validaciones, normalización de datos y cálculo de costos.
Repository	Accede a la base de datos usando interfaces que extienden JpaRepository.
Model	Define las entidades del dominio y sus relaciones JPA.

4. Modelo de datos

El modelo persistido está formado por tres entidades principales: Duenio, Mascota y Turno.

- Duenio: persona responsable de una o más mascotas.
- Mascota: paciente concreto de la veterinaria.
- Turno: atención programada para una mascota.

Las clases Persona y PacienteVeterinario son abstractas y usan `@MappedSuperclass`. No generan tablas propias, pero sus atributos se heredan en las entidades concretas.

Relaciones principales:

- Un Duenio puede tener muchas Mascota.
- Una Mascota pertenece a un Duenio.
- Una Mascota puede tener muchos Turno.
- Un Turno pertenece a una Mascota.

5. Funcionalidades principales

- CRUD de dueños.
- CRUD de mascotas asociadas a dueños.
- CRUD de turnos asociados a mascotas.
- Cálculo automático del costo estimado de un turno.
- Carga inicial de datos de ejemplo con `CommandLineRunner`.
- Consulta de datos desde frontend mediante llamadas `fetch`.

6. API REST principal

Recurso	Endpoints	Responsabilidad
Dueños	<code>/api/duenios</code>	Crear, listar, actualizar y eliminar dueños.
Mascotas	<code>/api/mascotas</code>	Crear, listar, actualizar y eliminar mascotas.
Turnos	<code>/api/turnos</code>	Crear, listar, actualizar y eliminar turnos.

7. Persistencia

La persistencia se implementa con Spring Data JPA. Cada repository extiende `JpaRepository`, lo que permite usar métodos como `findAll`, `findById`, `save` y `deleteById` sin escribir SQL manual para el CRUD básico.

Además, el proyecto incluye consultas derivadas por nombre de método, consultas JPQL con `@Query` y consultas SQL nativas con `nativeQuery = true`.

8. POO y patrón aplicado

- **Encapsulamiento:** atributos privados y acceso mediante getters/setters.
- **Herencia:** Duenio hereda de Persona; Mascota hereda de PacienteVeterinario.
- **Abstracción:** clases abstractas para representar conceptos generales.
- **Polimorfismo:** TurnoService usa varias implementaciones de CostoConsultaStrategy.
- **Strategy:** cada motivo de turno tiene una clase que calcula el costo correspondiente.

9. Flujo de creación de un turno

1. El usuario completa el formulario en el frontend.
2. `app.js` envía un `POST /api/turnos` con los datos del turno y el `mascotaId`.
3. `TurnoController` recibe el JSON y Spring lo convierte a `TurnoRequest`.
4. El controller busca la mascota por id y crea el objeto `Turno`.
5. `TurnoService` valida datos, normaliza motivo y estado, y usa `Strategy` para calcular el costo.
6. `TurnoRepository` guarda el turno en H2 mediante JPA/Hibernate.
7. Spring devuelve el turno guardado como JSON.
8. `app.js` recarga los datos y actualiza la tabla.

10. Ejecución

Comando principal:

```
.\mvnw.cmd spring-boot:run
```

URL de la aplicación:

```
http://localhost:8080
```

Consola H2:

```
http://localhost:8080/h2-console
```

11. Diagramas relacionados

- `docs/diagrama-clases-veterinaria.png`: diagrama de clases.
- `docs/DER-veterinaria.png`: diagrama entidad-relación.